Verifikation verteilter Systeme in Isabelle

Ein Vergleich von Kleppmann und I/O-Automaten

Sepp Stage

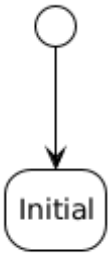
TU Chemnitz, Fakultät Betriebssysteme

TECHNISCHE UNIVERSITÄT
CHEMNITZ

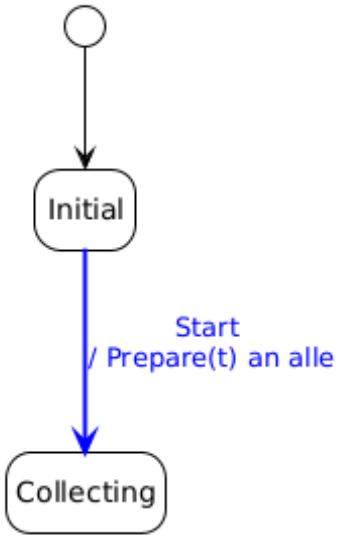
1. Einleitung
2. Das Zwei-Phase-Commit-Protokoll
    - Übergänge des Koordinators
    - Übergänge eines Teilnehmers
    - Beispielhafte Abläufe
3. I/O Automaten
    - Signatur
    - Mathematische Definition
    - Ausführung
    - Komposition
4. Vergleich
    - Modellierung
    - Invarianten
    - Beweis einer Invarianten
    - Beweis der einheitlichen Zustimmung
5. Vergleich
    - Kriterien...
6. Fazit

AT&T

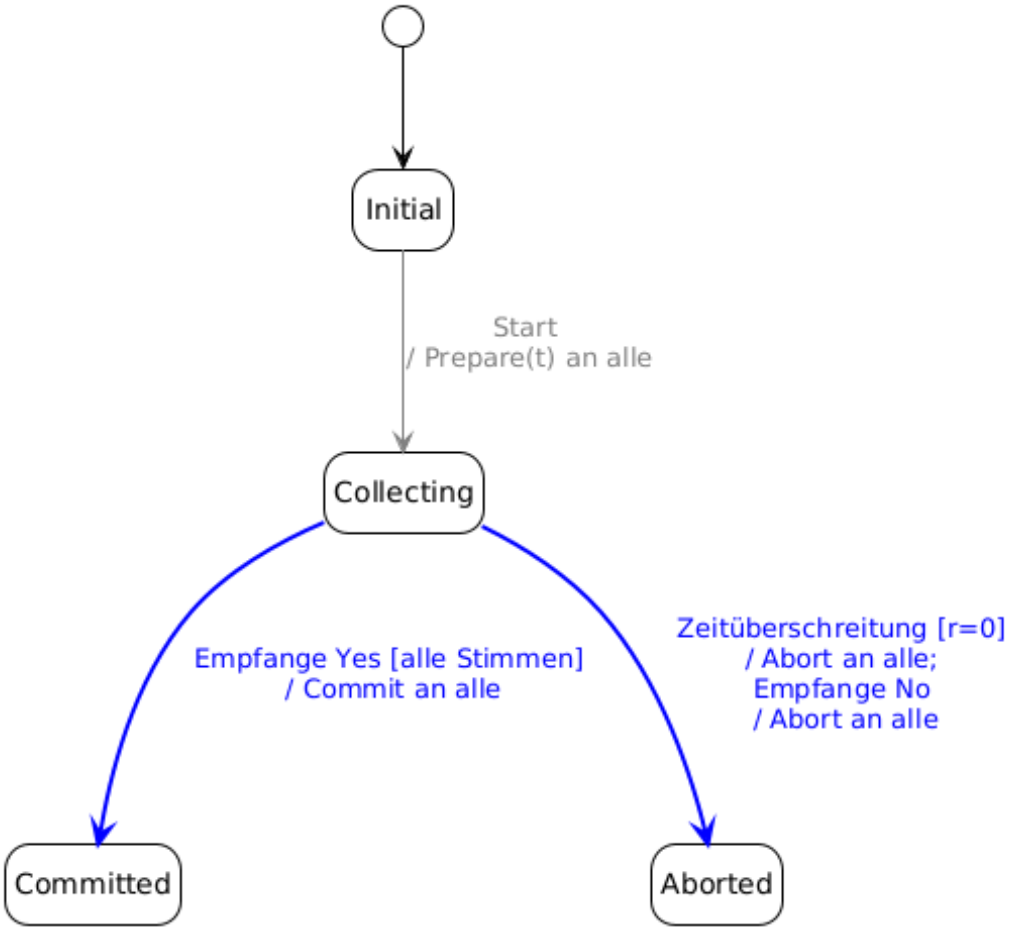
Das Zwei-Phase-Commit-Protokoll
Übergänge des Koordinators



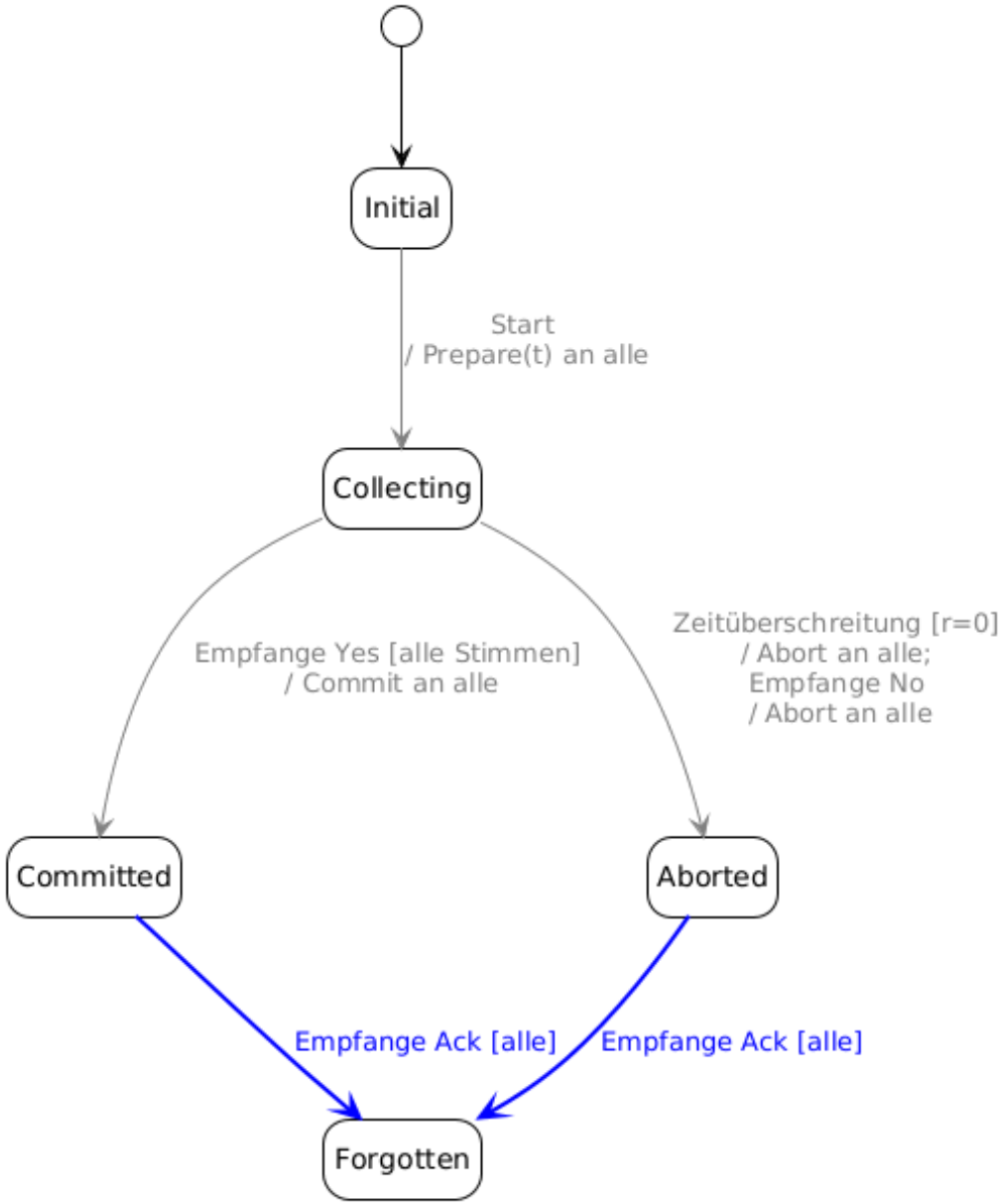
Das Zwei-Phase-Commit-Protokoll
Übergänge des Koordinators



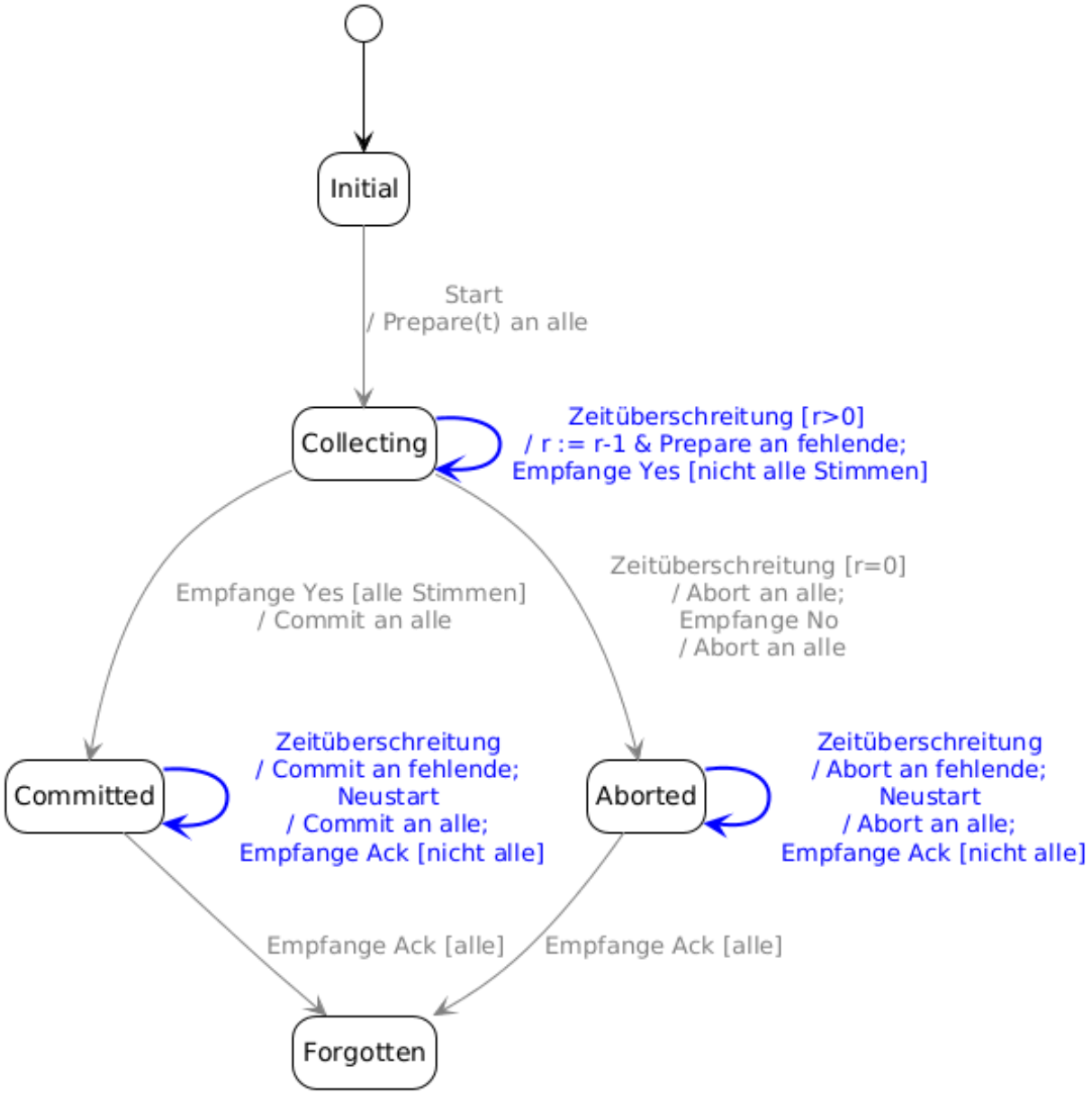
Das Zwei-Phase-Commit-Protokoll
Übergänge des Koordinators



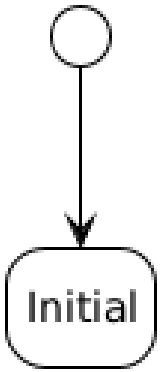
Das Zwei-Phase-Commit-Protokoll
Übergänge des Koordinators



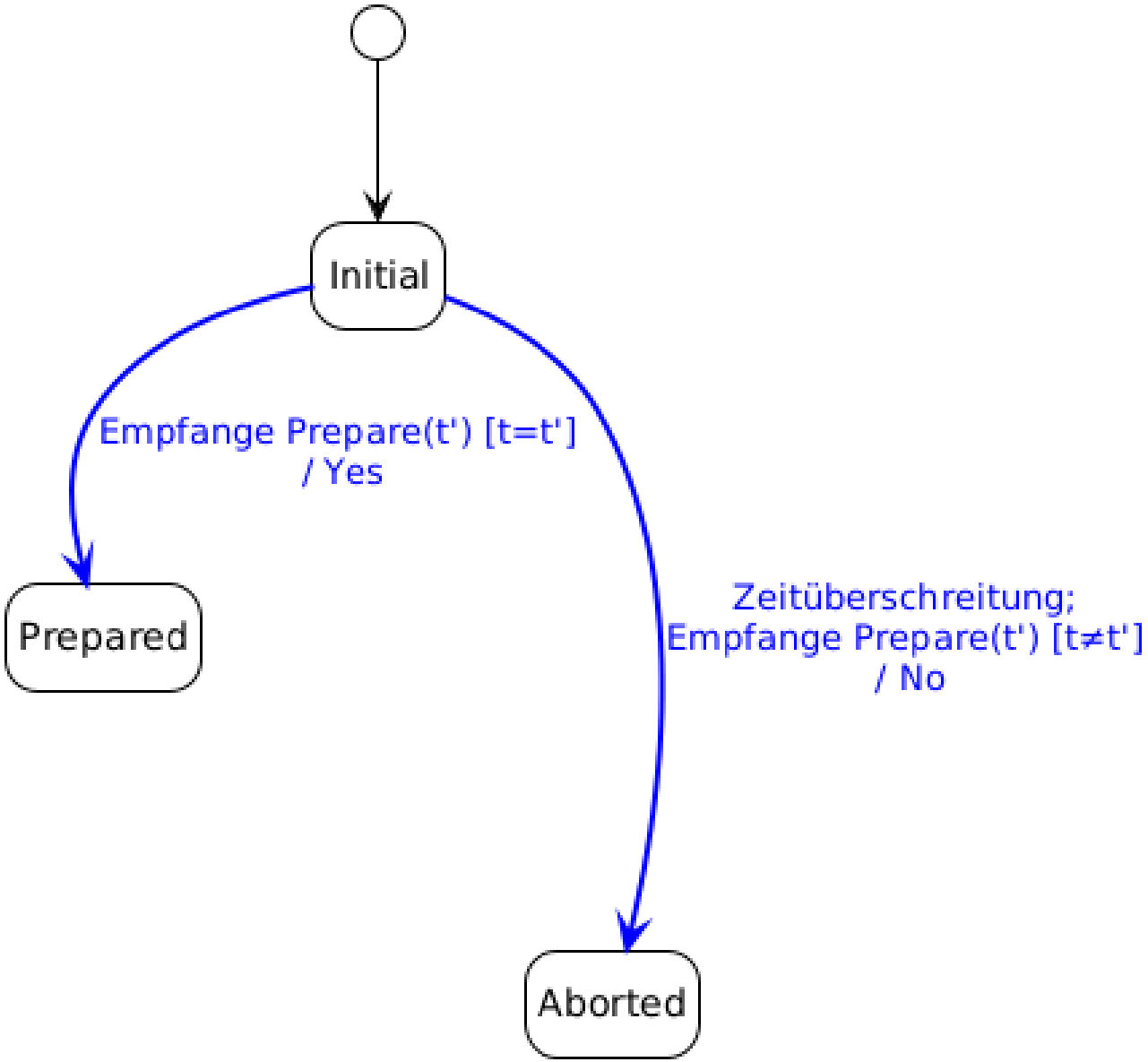
Das Zwei-Phase-Commit-Protokoll
Übergänge des Koordinators



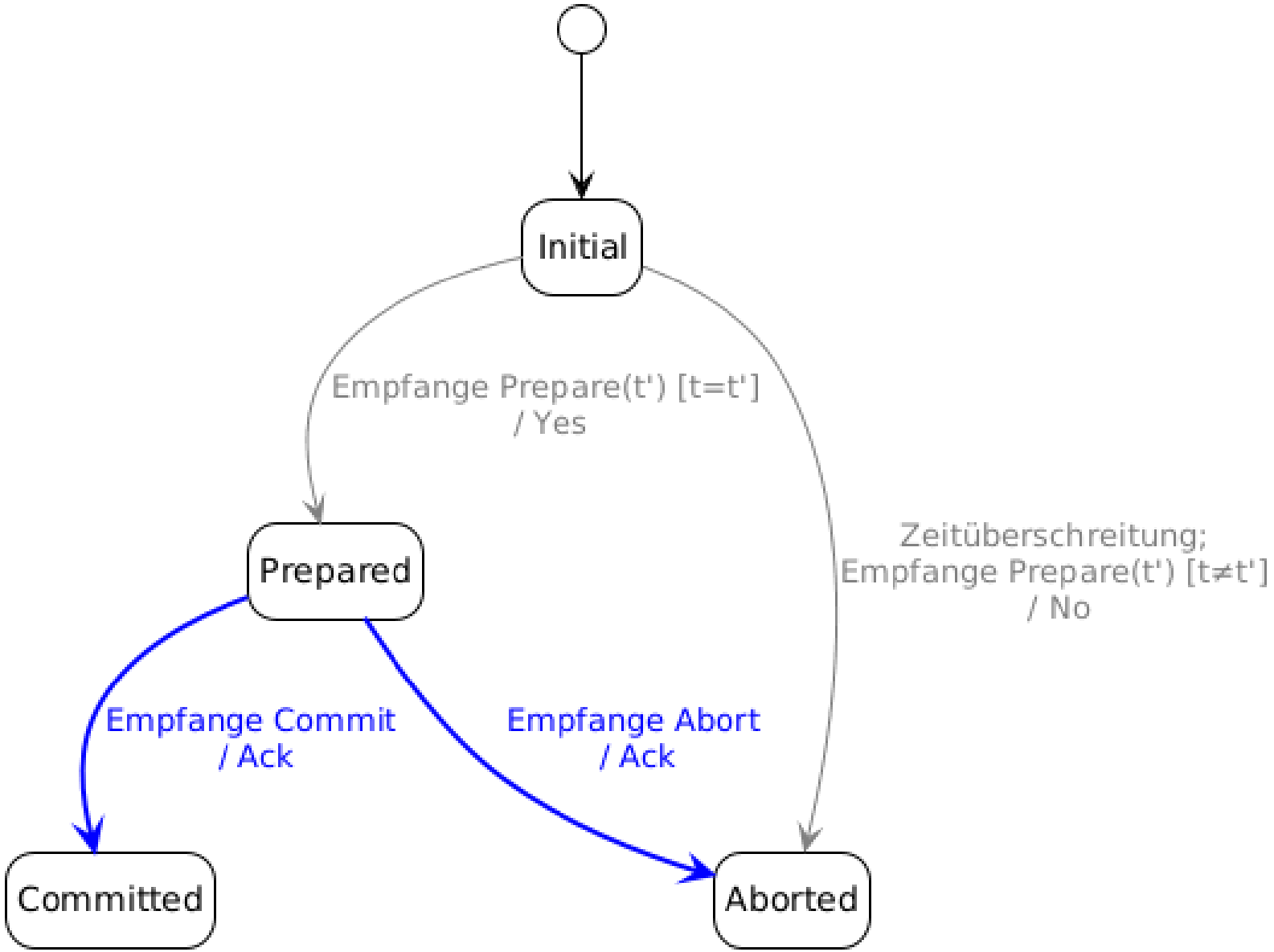
Das Zwei-Phase-Commit-Protokoll
Übergänge eines Teilnehmers



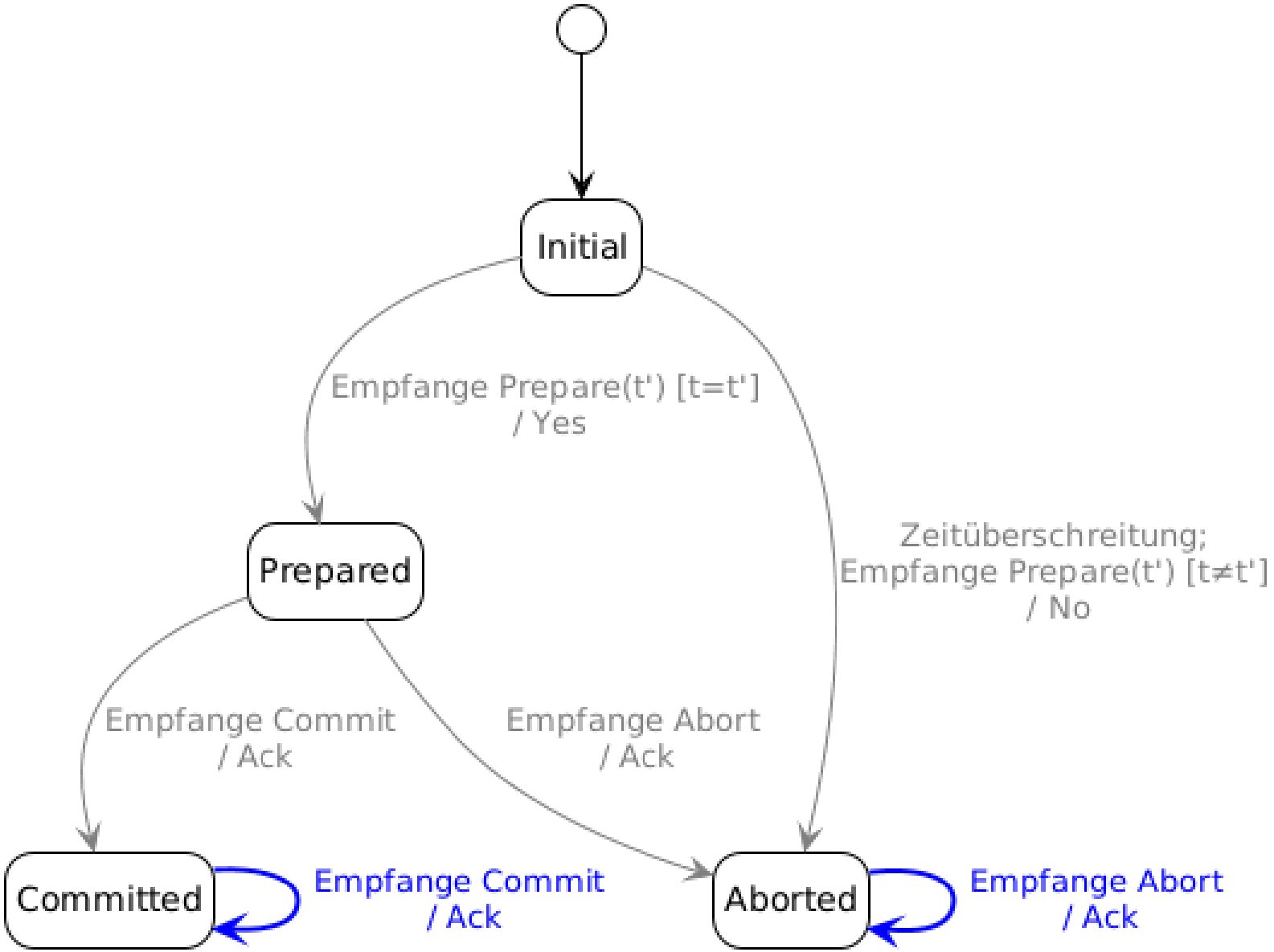
Das Zwei-Phase-Commit-Protokoll
Übergänge eines Teilnehmers



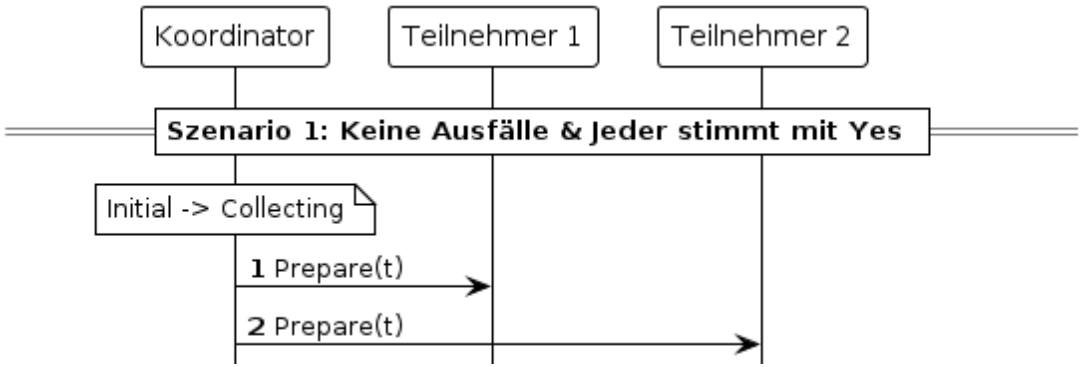
Das Zwei-Phase-Commit-Protokoll
Übergänge eines Teilnehmers



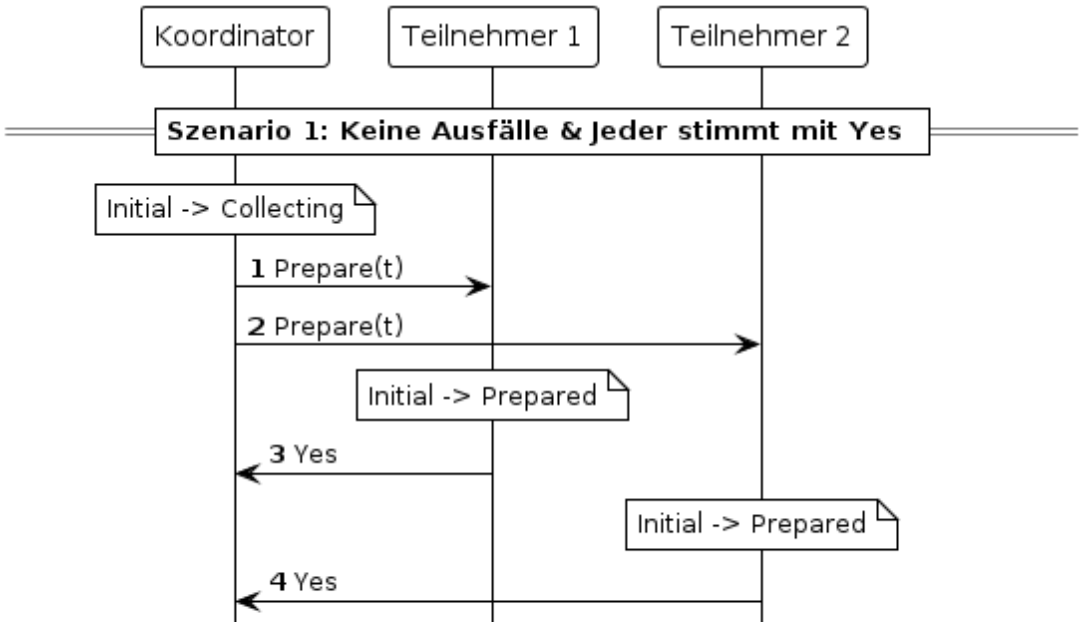
Das Zwei-Phase-Commit-Protokoll
Übergänge eines Teilnehmers



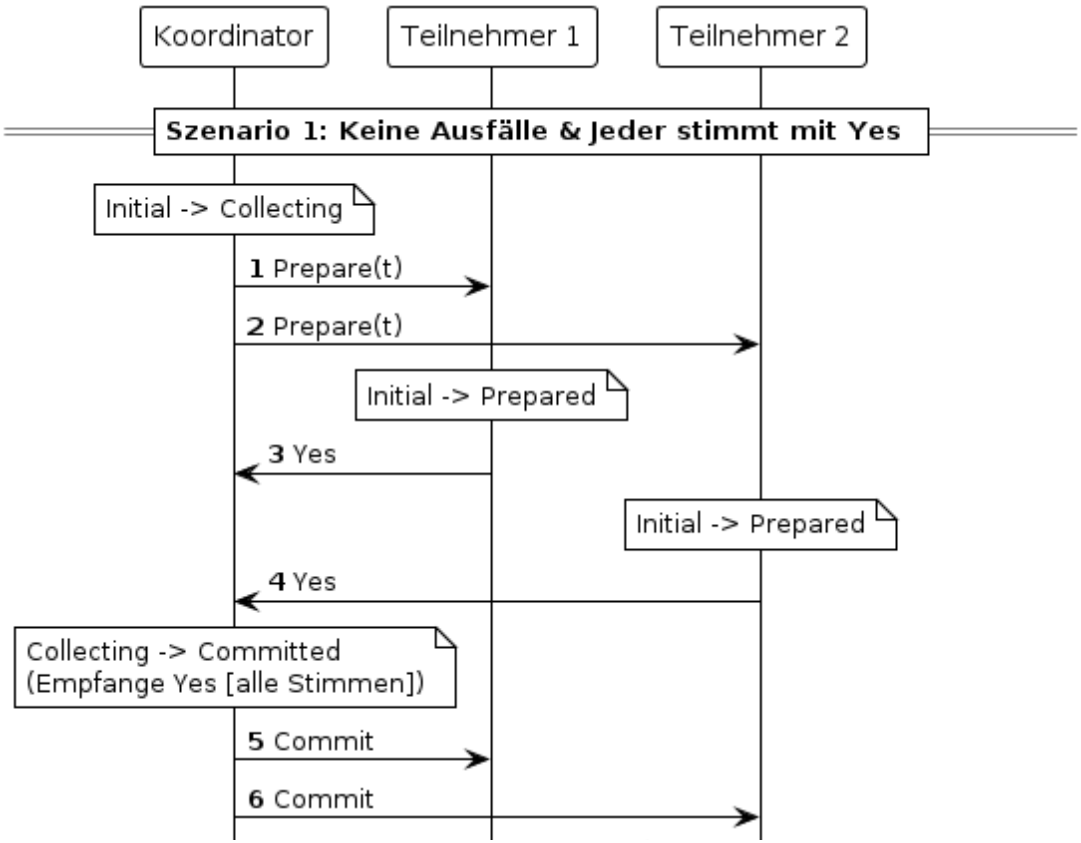
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



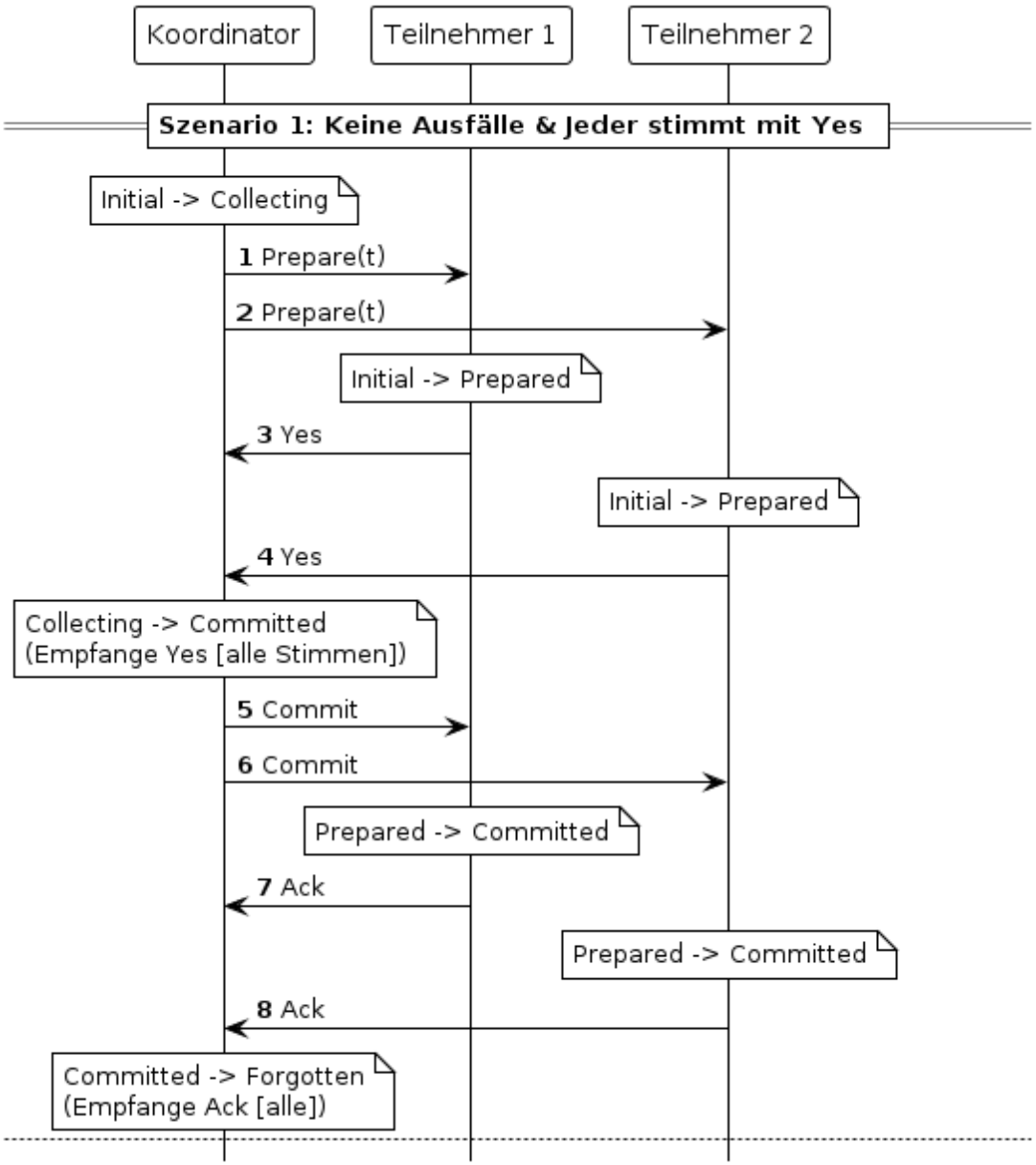
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



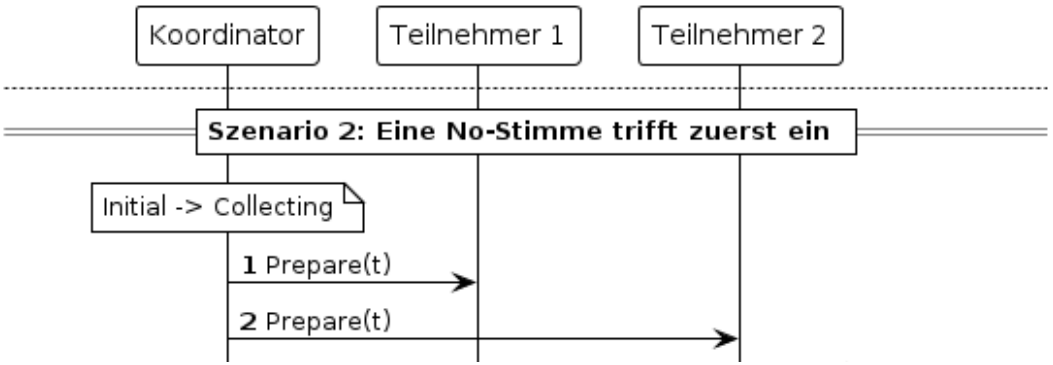
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



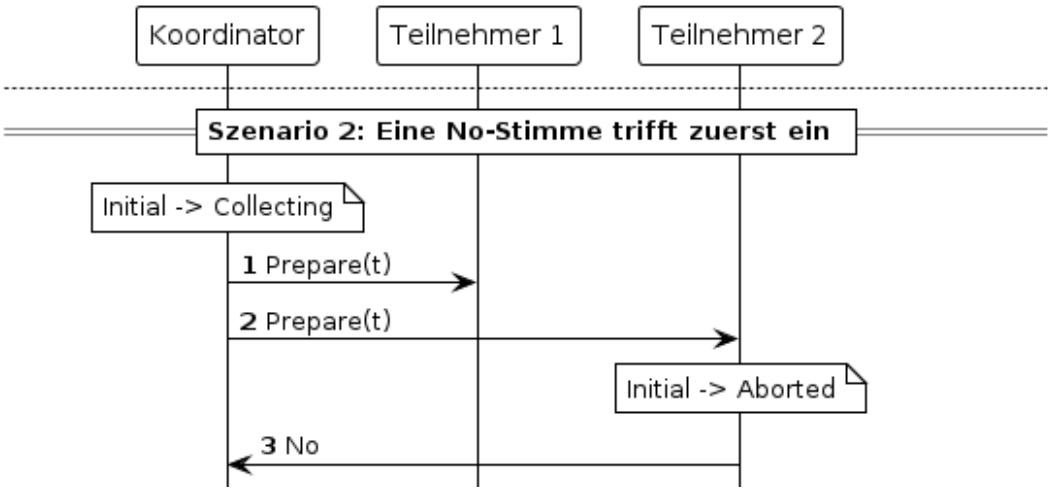
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



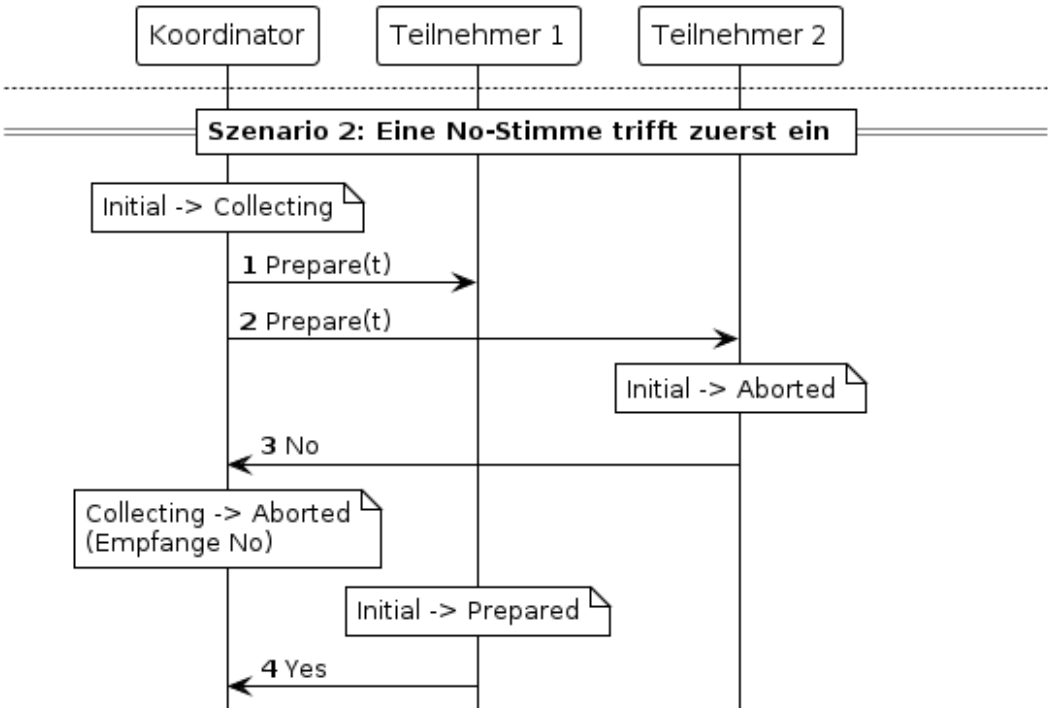
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



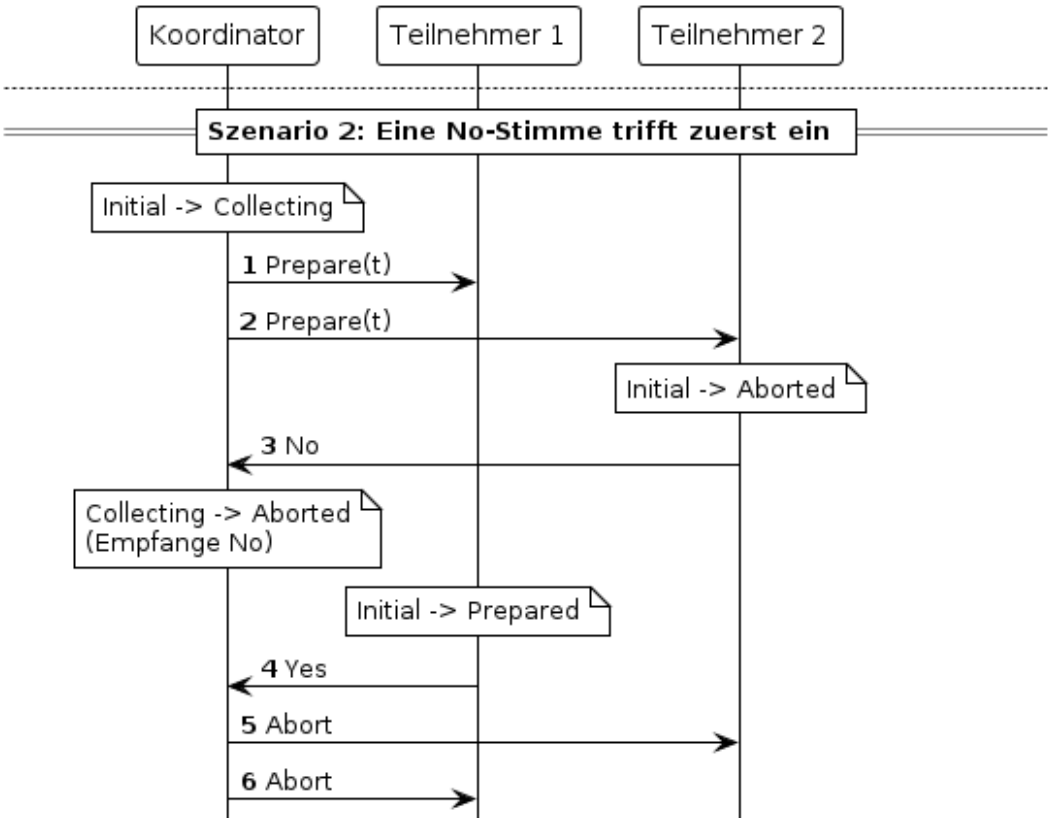
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



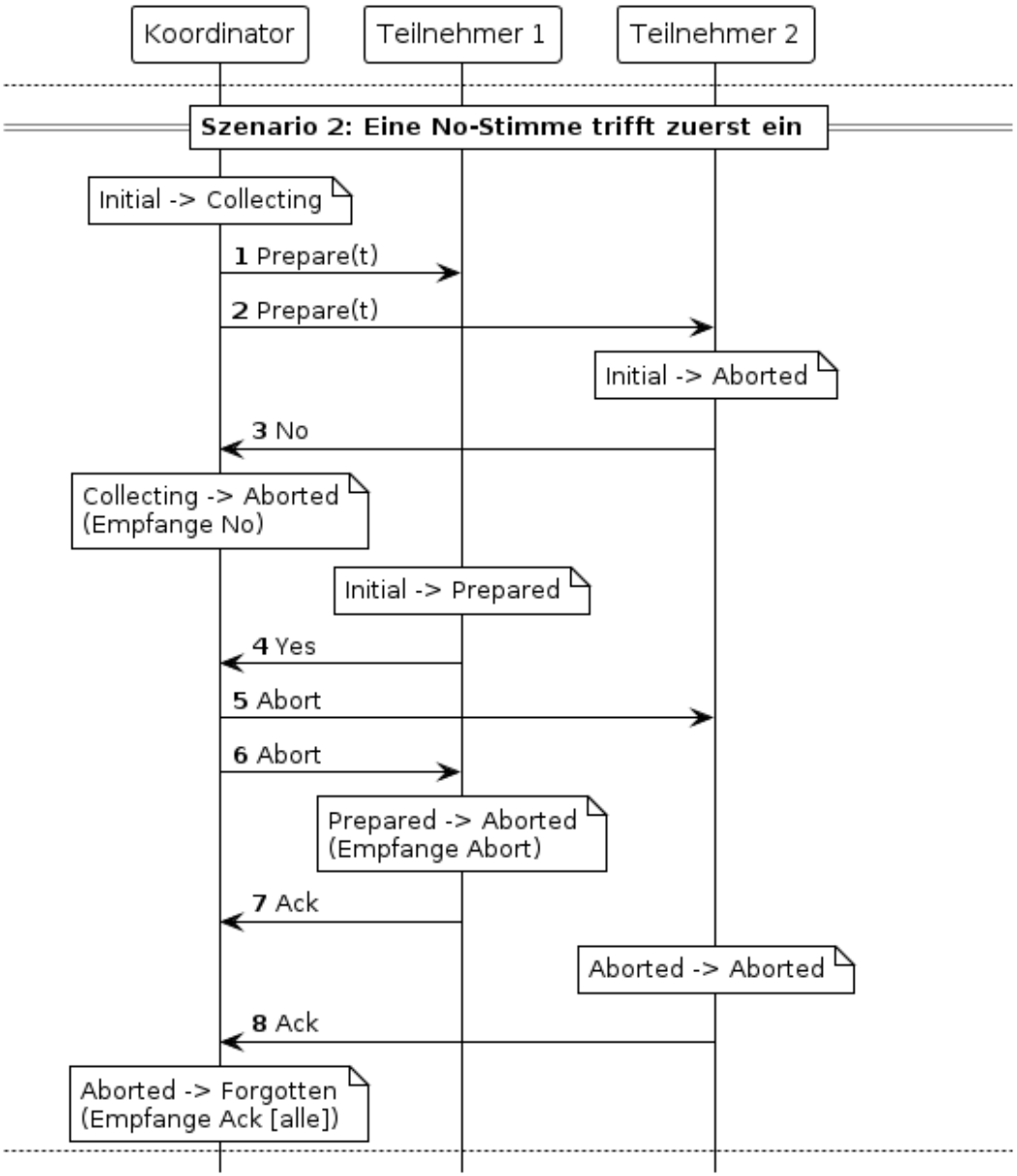
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



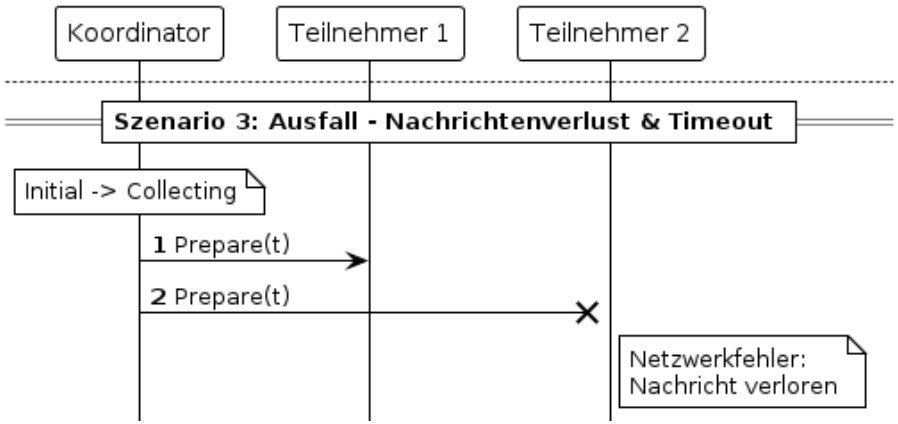
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



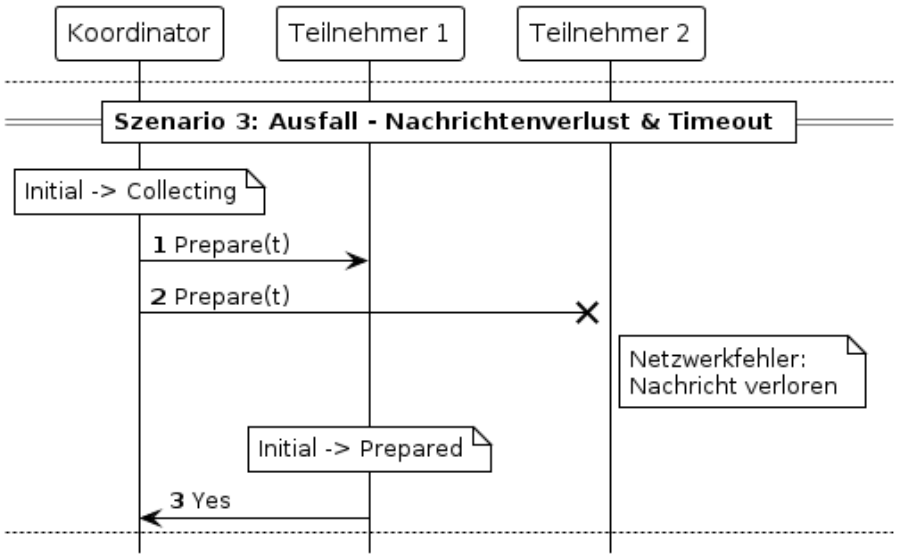
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



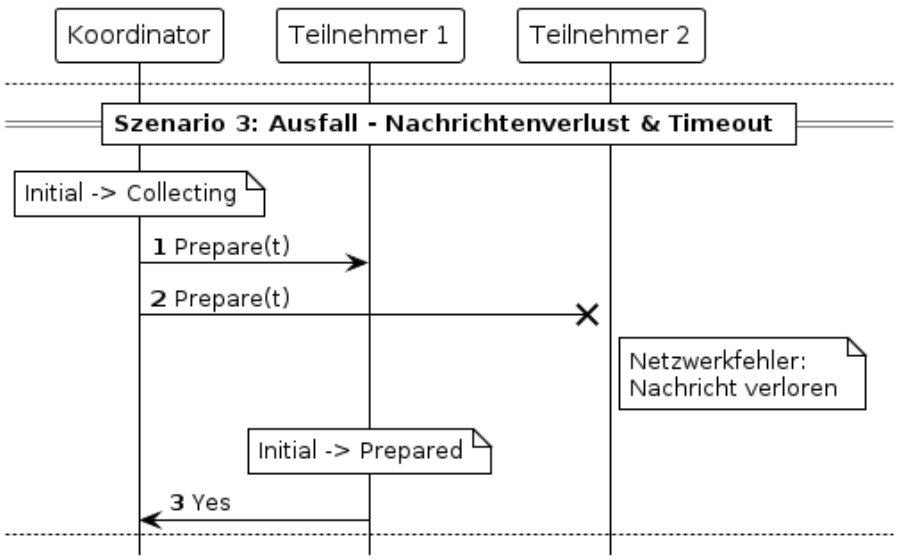
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe

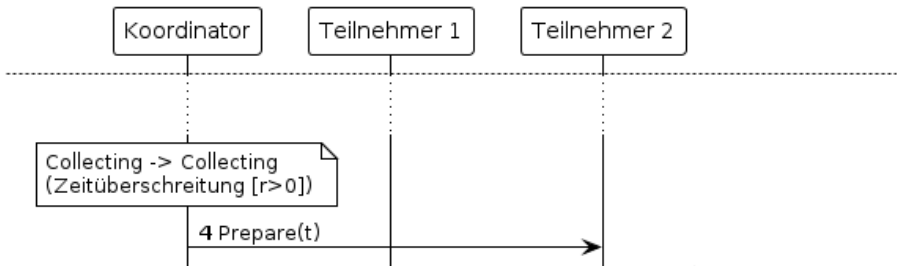


Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe

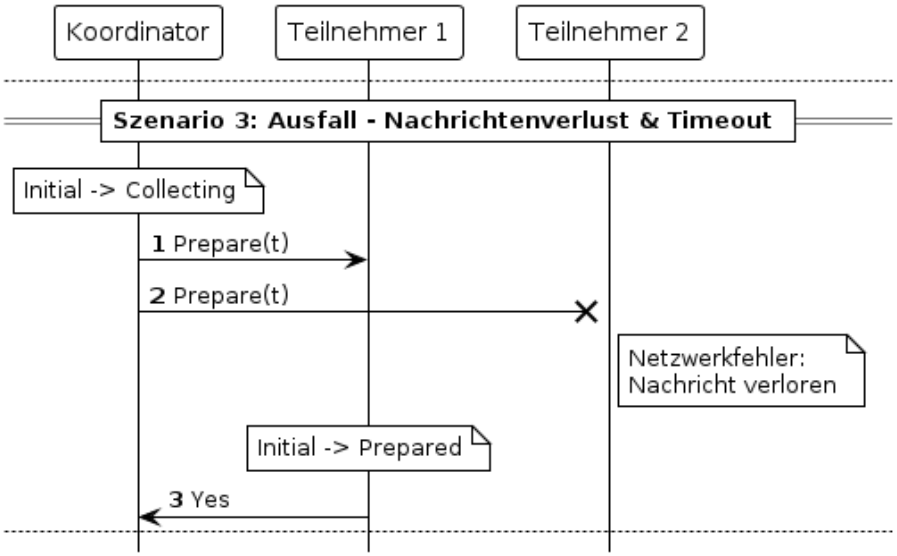


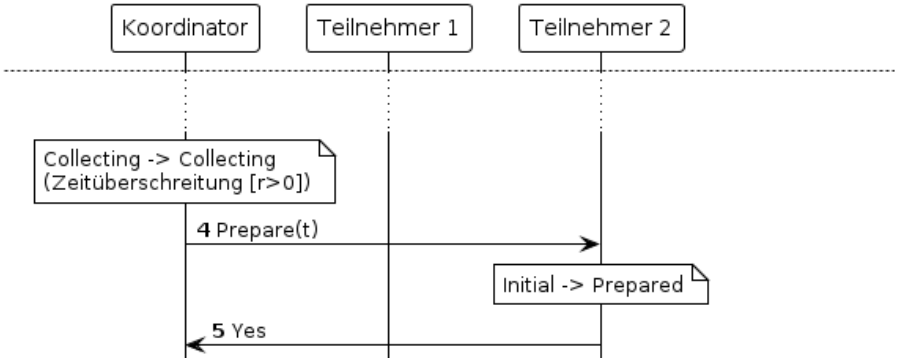
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



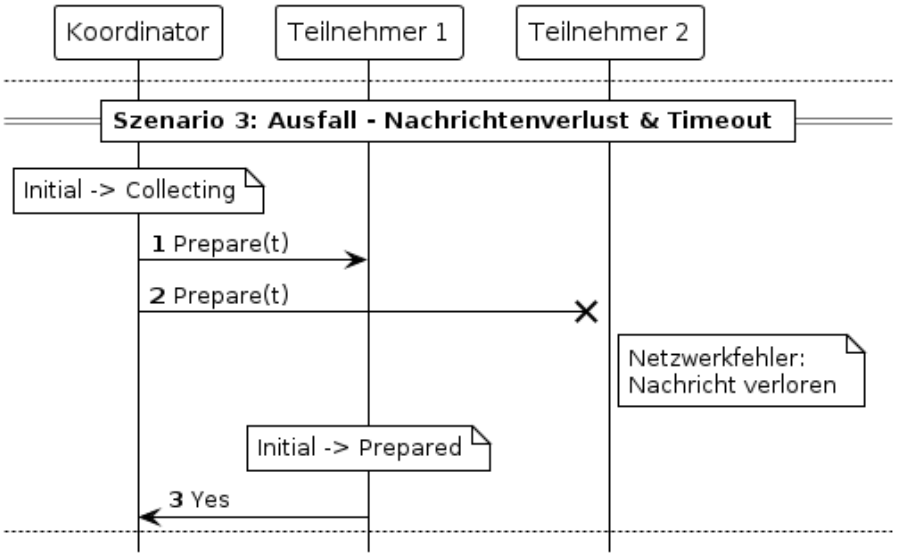


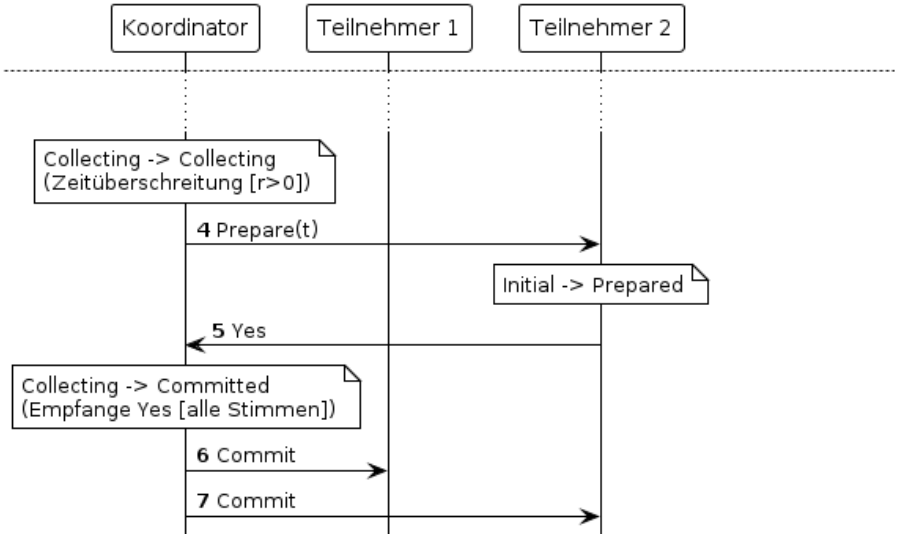
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



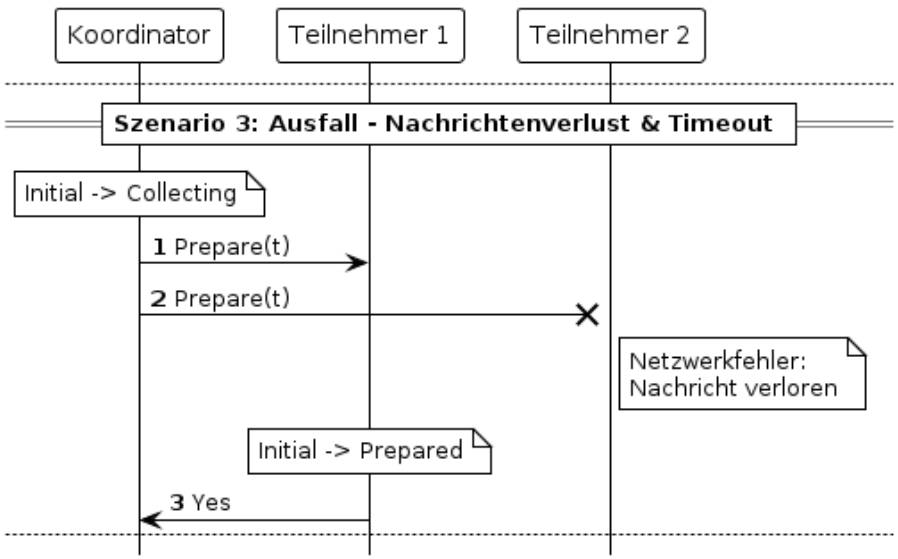


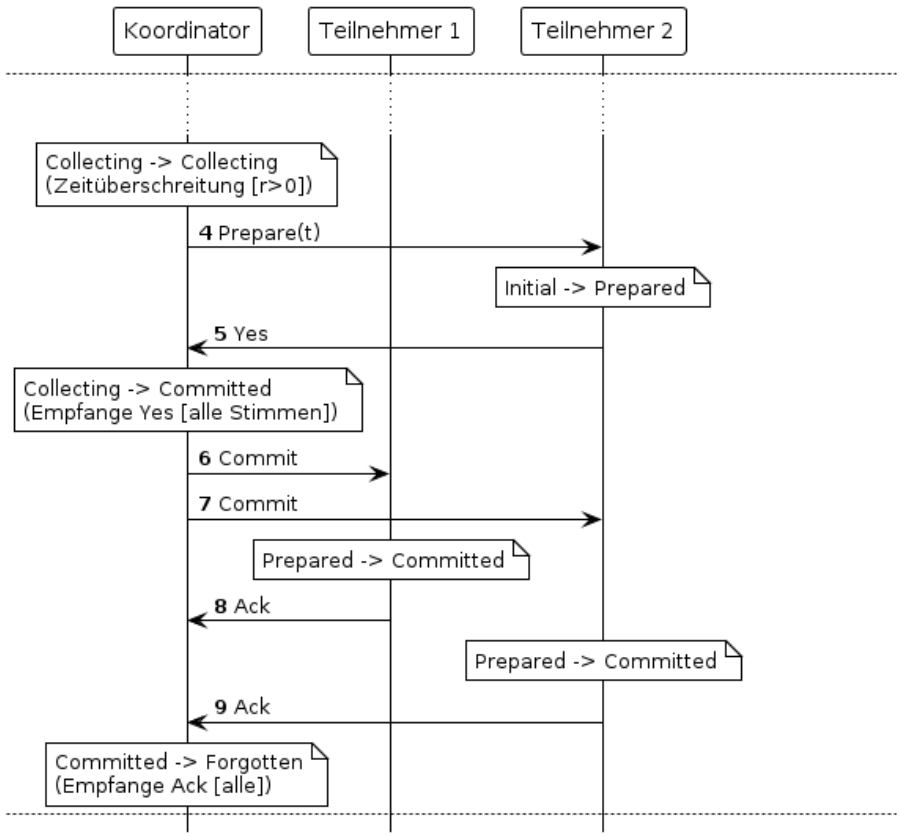
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



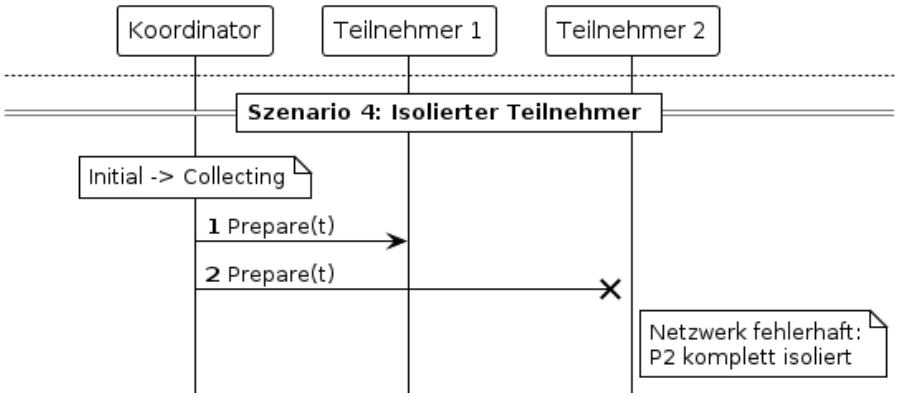


Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe

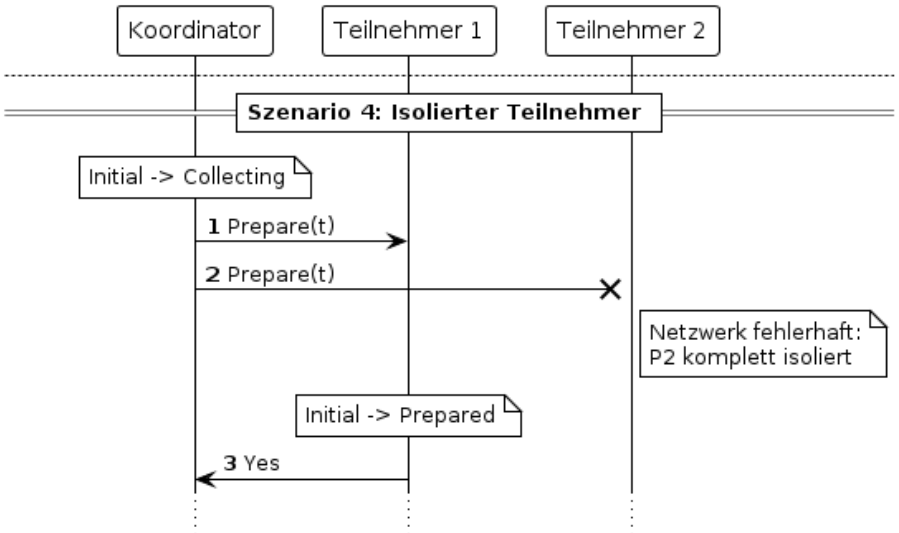




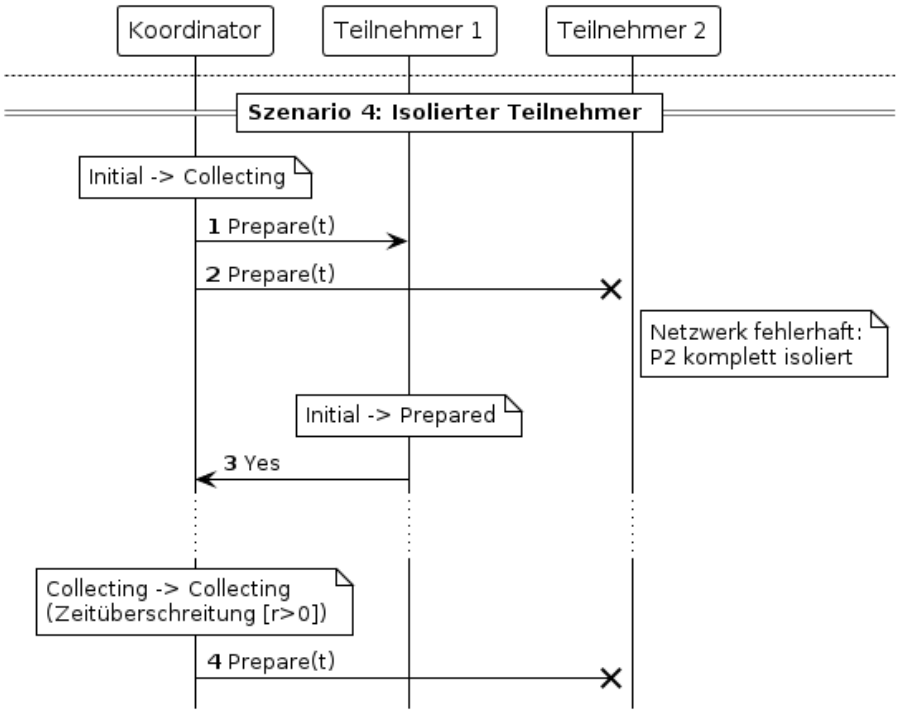
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



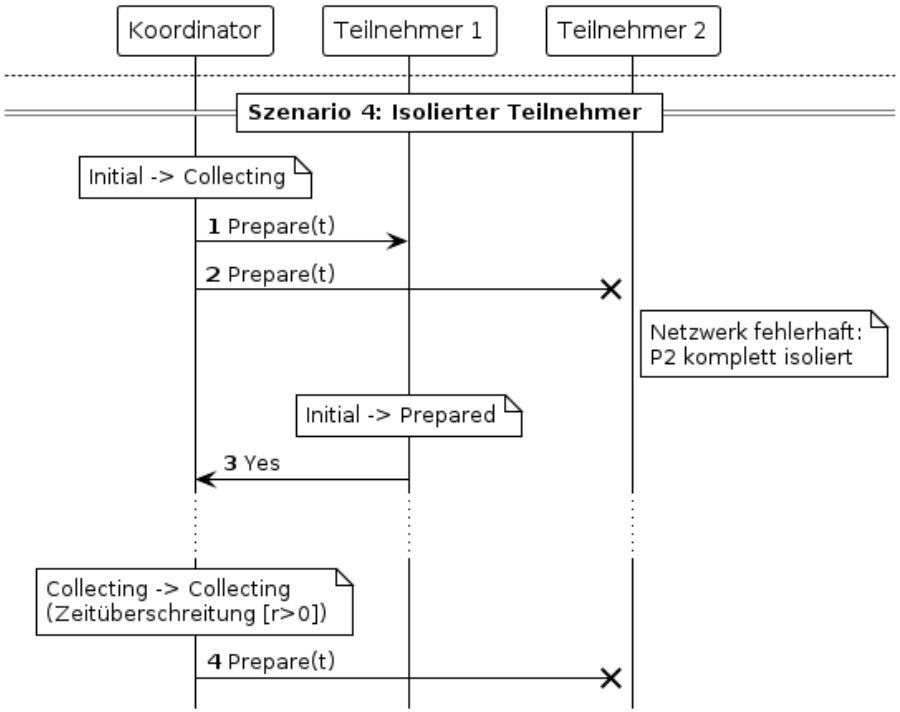
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe

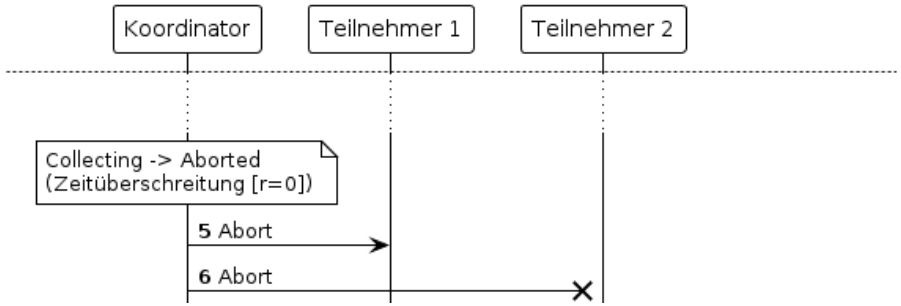


Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe

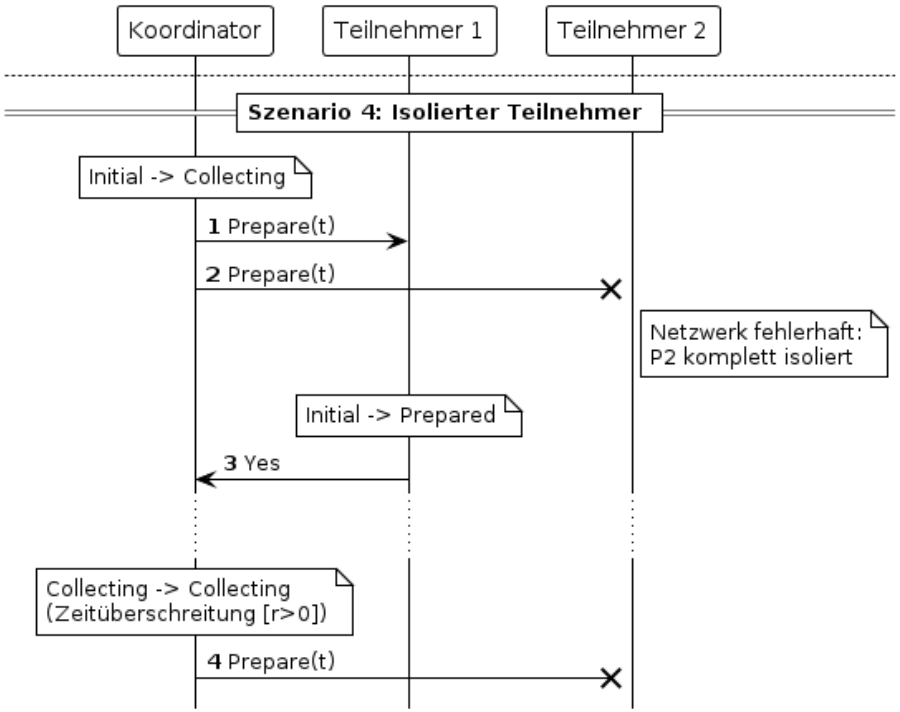


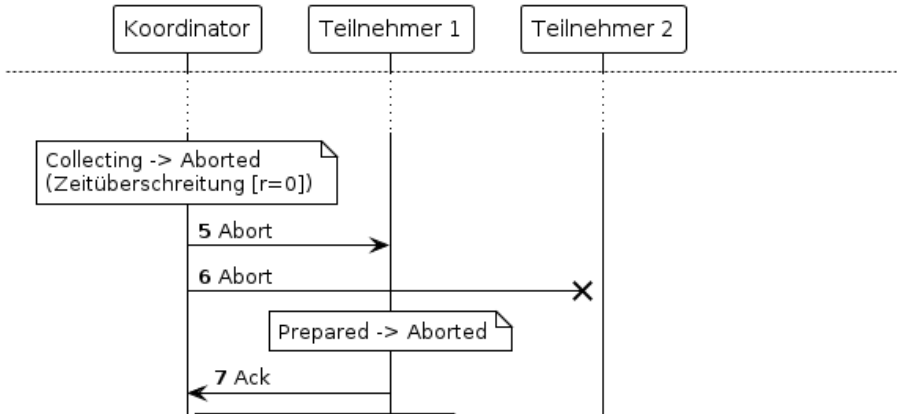
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



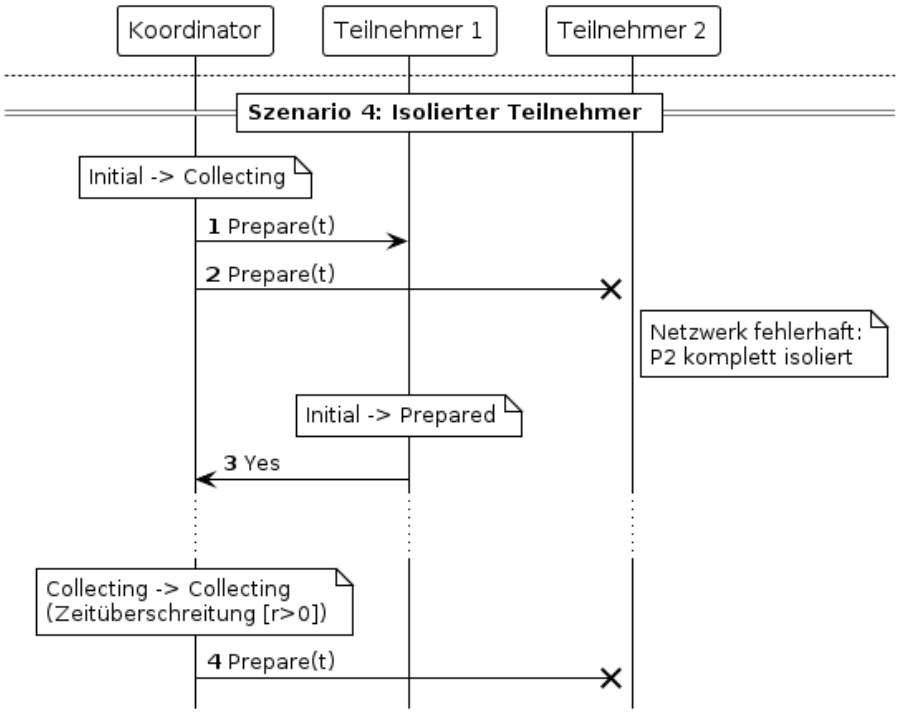


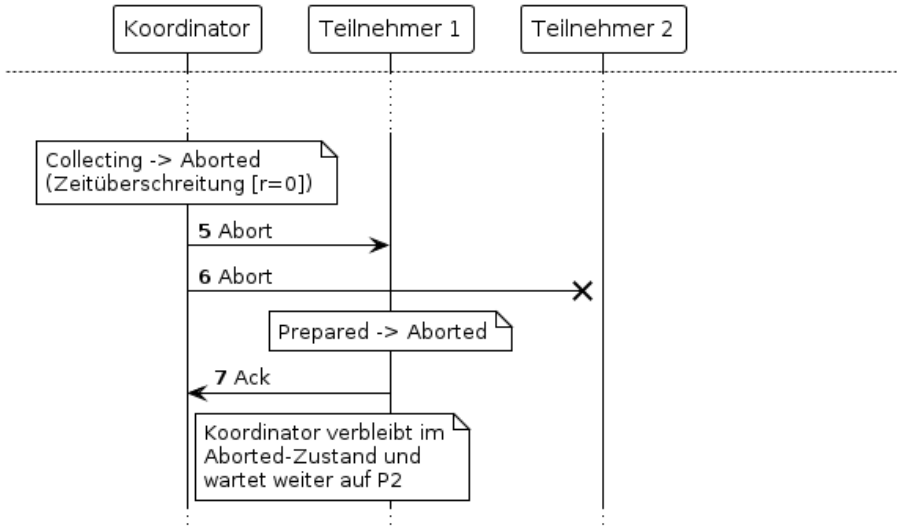
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



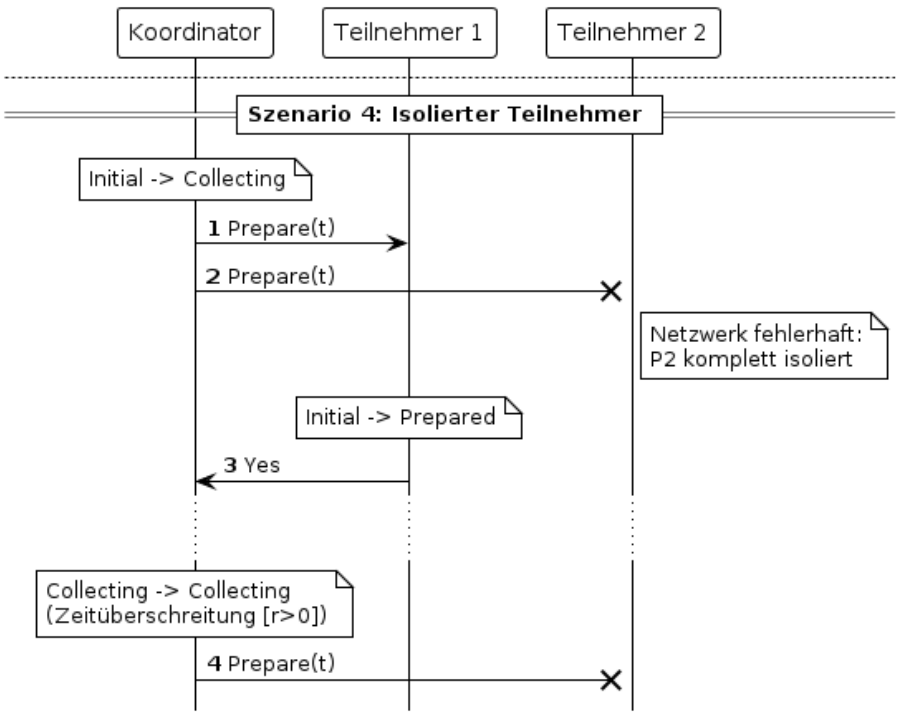


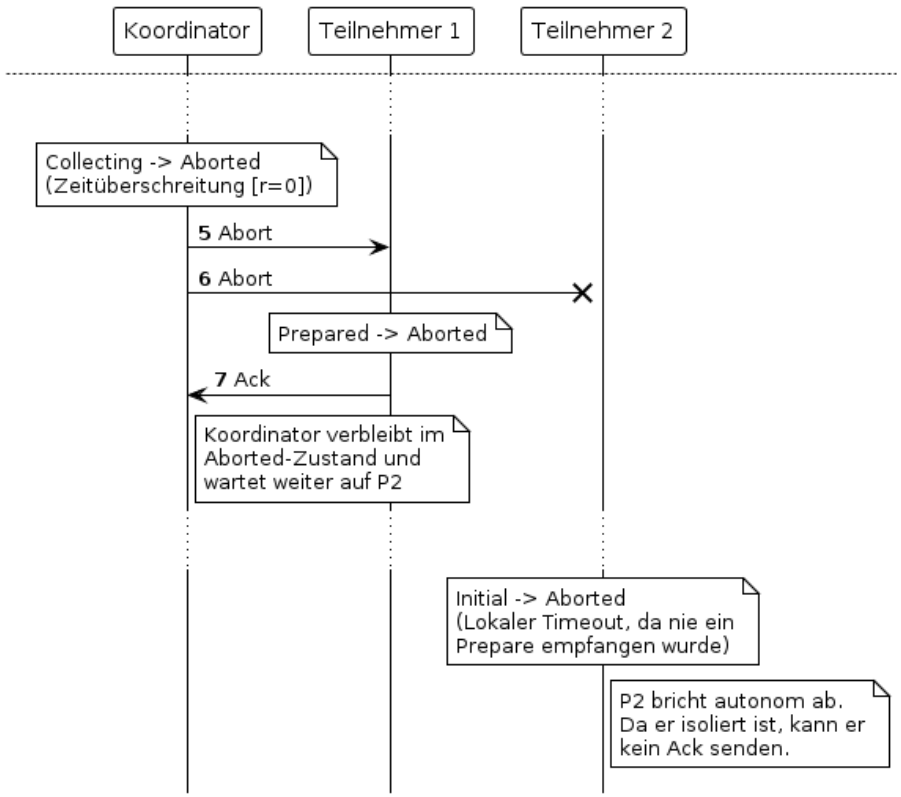
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe





Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe





Definition einer Signatur S

- *Input-Übergänge* $in(S)$
- *Output-Übergänge* $out(S)$
- *Internal-Übergänge* $int(S)$

Beispielbild!!!!!

Definition eines I/O Automaten A

- *Signatur* $sig(A)$
- *Menge an Zuständen* $states(A)$
- *nichtleere Menge an Startzuständen* $start(A) \subseteq states(A)$
- *Übergangsrelation*

$$trans(A) \subseteq states(A) \times acts(sig(A)) \times states(A)$$

- *Aufgabenpartition* $tasks(A)$

Beispielbild????

Ausführung eines I/O Automaten A

- *Ausführungsfragment* $s_0, \pi_1, s_1, \pi_2, ...$
- $\forall i \geq 0.(s_i, \pi_{i+1}), s_{i+1}) \in trans(A)$
- *Ausführung falls* $s_0 \in starts(A)$

Beispielbild????

Komposition zweier I/O Automaten A & B

- *Automaten müssen kompatibel sein*
- *Signaturen werden Verknüpft*
- *entstehende Zustände sind Vektoren* (s_a, s_b) *mit* $s_i \in start(I)$
- A *und* B *dürfen bei gemeinsamen Übergängen gleichzeitig agieren*

OUTPUTS bleiben OUTPUTS für weitere Komposition!

```
datatype 't msg = Prepare (transaction: 't) | Yes | No |
  ↪  Commit | Abort | Ack
datatype ('proc, 'msg) event
  = Start
  | Receive (msg_sender: 'proc) (recv_msg: 'msg)
  | Timeout
  | Restart
```

```
datatype 't msg = Prepare (transaction: 't) | Yes | No |
  ↪  Commit | Abort | Ack
datatype ('proc, 'msg) action =
    Start "'proc"
  | Send "'proc" "'proc" "'msg"
  | Receive "'proc" "'proc" "'msg"
  | Timeout "'proc"
  | Restart "'proc"
```

```
50  definition coord_asig :: "'proc ⇒ ('proc,'t) action
     ↪  signature" where
51  "coord_asig pid =
52   ({Receive q pid m | q m. True} ∪ {Start pid, Timeout pid,
      ↪  Restart pid},
53    {Send pid q m | q m. True},
54    {})"
89  definition parts_asig :: "'proc set ⇒ ('proc,'t msg) action
     ↪  signature" where
90  "parts_asig P =
91   ({Receive q p m | q p m. p ∈ P} ∪ {Timeout p | p. p ∈ P} ∪
      ↪  {Restart p | p. p ∈ P},
92    {Send p q m | p q m. p ∈ P},
93    {})"
113 definition chan_asig :: "('proc,'t) action signature" where
114 "chan_asig =
115  ({Send s r m | s r m. True},
116   {Receive s r m | s r m. True},
117   {})"
```

```
7  datatype ('t, 'proc) state = Initial (transaction: 't) (all:
   ↪  "'proc set") (retry_count:"nat")
8    | Collecting (transaction: 't) (all: "'proc set")
   ↪  (yes:"'proc set") (retry_count:"nat")
9    | Prepared
10   | Committed (all: "'proc set") (ack: "'proc set")
11   | Aborted (all: "'proc set") (ack: "'proc set")
12   | Forgotten
```

```
17 datatype ('t,'proc) cstate = CInitial (transaction: 't)
   ↪  (all: "'proc set") (retry_count:"nat")
18   | Collecting (transaction: 't) (all: "'proc set")
   ↪  (yes:"'proc set") (retry_count:"nat")
19   | CCommitted (all: "'proc set") (ack: "'proc set")
20   | CAborted (all: "'proc set") (ack: "'proc set")
21   | Forgotten
22 datatype ('t,'proc) coord_state = CState "('t,'proc) cstate"
   ↪  "('proc × 'proc × 't msg) set"
```

```
7  datatype ('t, 'proc) state = Initial (transaction: 't) (all:
   ↪  "'proc set") (retry_count:"nat")
8    | Collecting (transaction: 't) (all: "'proc set")
   ↪  (yes:"'proc set") (retry_count:"nat")
9    | Prepared
10   | Committed (all: "'proc set") (ack: "'proc set")
11   | Aborted (all: "'proc set") (ack: "'proc set")
12   | Forgotten
```

```
61 datatype 't pstate = PInitial (transaction: 't)
62   | Prepared
63   | PCommitted
64   | PAborted
65 datatype ('t,'proc) part_state = PState "'t pstate" "('proc
   ↪  × 'proc × 't msg) set"
```

```
7   datatype ('t, 'proc) state = Initial (transaction: 't) (all:
    ↪  "'proc set") (retry_count:"nat")
8     | Collecting (transaction: 't) (all: "'proc set")
      ↪  (yes:"'proc set") (retry_count:"nat")
9     | Prepared
10    | Committed (all: "'proc set") (ack: "'proc set")
11    | Aborted (all: "'proc set") (ack: "'proc set")
12    | Forgotten
```

```
106 type_synonym ('proc,'t) chan_state = "('proc × 'proc × 't
    ↪  msg) set"
```

```
14  fun coordinator_step::
15      "'proc ⇒ ('t, 'proc) state ⇒ ('proc, 't msg) event ⇒
        ↪  ('t, 'proc) state × ('proc, 't msg) send set"  where
34  fun participant_step::
35      "('t, 'proc) state ⇒ ('proc, 't msg) event ⇒ ('t,
        ↪  'proc) state × ('proc, 't msg) send set" where
```

```
24  fun coordinator_step::
25      "'proc ⇒ ('t, 'proc) cstate ⇒ ('proc, 't msg) action ⇒
        ↪  ('t, 'proc) cstate × ('proc × 't msg) set"  where
67  fun participant_step::
68      "'proc ⇒ 't pstate ⇒ ('proc, 't msg) action ⇒ 't pstate
        ↪  × ('proc × 't msg) set"  where
```

```
16  ‹coordinator_step pid (Initial t a r) Start = (Collecting t
    ↪  a {pid} r, {Send p (Prepare t) | p. p ∈ a})› |
```

```
26  ‹coordinator_step pid (CInitial t a r) (Start _) =
    ↪  (Collecting t a {pid} r, {(q, (Prepare t)) | q. q ∈ a -
    ↪  {pid}})› |
```

```
17    ‹coordinator_step pid (Collecting t a y r) (Receive sender
      ↪  msg) =
18      (case msg of
19        Yes ⇒(if y ∪ {sender} = a then (Committed a {pid},
          ↪  {Send p (Commit) | p. p ∈ a}) else (Collecting t
          ↪  a (y ∪ {sender}) r, {})) |
20        No ⇒(Aborted a {pid}, {Send p (Abort) | p. p ∈ a}) |
21        _ ⇒(Collecting t a y r, {}))› |
```

```
27    ‹coordinator_step pid (Collecting t a y r) (Receive sender
      ↪  _ msg) =
28      (case msg of
29        Yes ⇒(if y ∪ {sender} = a then (CCommitted a {pid},
          ↪  {(q, Commit) | q. q ∈ a - {pid}}) else
          ↪  (Collecting t a (y ∪ {sender}) r, {})) |
30        No ⇒(CAborted a {pid}, {(q, Abort) | q. q ∈ a -
          ↪  {pid}}) |
31        _ ⇒(Collecting t a y r, {}))› |
```

```
22    ‹coordinator_step pid (Collecting t a y 0) Timeout =
      ↪  (Aborted a {pid}, {Send p (Abort) | p. p ∈ a})› |
23    ‹coordinator_step _ (Collecting t a y (Suc r)) Timeout =
      ↪  (Collecting t a y r, {Send p (Prepare t) | p. p ∈ (a -
      ↪  y)})› |
```

```
32    ‹coordinator_step pid (Collecting t a y 0) (Timeout _) =
      ↪  (CAborted a {pid}, {(q, Abort) | q. q ∈ a - {pid}})› |
33    ‹coordinator_step pid (Collecting t a y (Suc r)) (Timeout
      ↪  _) = (Collecting t a y r, {(q, Prepare t) | q. q ∈ (a -
      ↪  y)})› |
```

```
26    ‹coordinator_step _ (Committed a ack') Timeout = (Committed
      ↪  a ack', {Send p Commit | p. p ∈ (a - ack')})› |
27    ‹coordinator_step _ (Committed a ack') Restart = (Committed
      ↪  a ack', {Send p Commit | p. p ∈ a})› |
```

```
36    ‹coordinator_step pid (CCommitted a ack') (Timeout _) =
      ↪  (CCommitted a ack', {(q, Commit) | q. q ∈ (a - ack')})›
      ↪  |
37    ‹coordinator_step pid (CCommitted a ack') (Restart _) =
      ↪  (CCommitted a ack', {(q, Commit) | q. q ∈ a - {pid}})›
      ↪  |
```

```
44  definition coord_trans ::
45    "'proc ⇒ (('t,'proc) coord_state × ('proc,'t msg) action ×
      ↪  ('t,'proc) coord_state) set" where
46  "coord_trans pid =
47    {(CState s msgs, a, CState s' (msgs ∪ ((λmsg. (pid, msg)) `
      ↪  sent))) | s a s' msgs sent. coordinator_step pid s a =
      ↪  (s', sent) ∧ (∃s m. a = Start pid ∨ a = Receive s pid m
      ↪  ∨ a = Timeout pid ∨ a = Restart pid)}
48    ∪ {(CState s msgs, Send pid rcpt msg, CState s (msgs -
      ↪  {(pid,rcpt,msg)})) | s msgs rcpt msg. (pid,rcpt,msg) ∈
      ↪  msgs}"

56  definition Coordinator :: "'p ⇒ 't ⇒ 'p set ⇒ nat ⇒
    ↪  (('p,'t msg) action, ('t,'p) coord_state) ioa" where
57  "Coordinator pid t a r = (coord_asig pid, {CState (CInitial t
    ↪  a r) {}}, coord_trans pid, {}, {})"
```

```
95  definition parts_trans :: "'proc set ⇒ (('proc ⇒ ('t,'proc)
    ↪  part_state) × ('proc,'t msg) action × ('proc ⇒
    ↪  ('t,'proc) part_state)) set" where
96  "parts_trans P = { (s, a, s').
97      ∀p. if p ∈ P ∧ ((∃q m. a = Receive q p m) ∨ a = Timeout p
        ↪  ∨ a = Restart p ∨ (∃q m. a = Send p q m))
98          then (s p, a, s' p) ∈ part_trans p
99          else s' p = s p }"
100
101 definition Participants :: "'proc set ⇒ ('proc ⇒ 't) ⇒
    ↪  (('proc,'t msg) action, 'proc ⇒ ('t,'proc) part_state)
    ↪  ioa" where
102 "Participants P t = (parts_asig P, {λp. PState (PInitial (t
    ↪  p)) {}}, parts_trans P, {}, {})"
```

```
108 definition chan_trans :: "(('proc,'t) chan_state × ('proc,'t
    ↪  msg) action × ('proc,'t) chan_state) set" where
109 "chan_trans =
110   {(M, Send s r m, M ∪ {(s,r,m)}) | M s r m. True}
111   ∪ {(M, Receive s r m, M) | M s r m. (s,r,m) ∈ M}"
119 definition Channel :: "(('proc,'t msg) action, ('proc,'t)
    ↪  chan_state) ioa" where
120 "Channel = (chan_asig, { {} }, chan_trans, {}, {})"
```

```
47  fun tupac_step where
48    ‹tupac_step proc = (if proc = 0 then coordinator_step proc
      ↪  else participant_step)›
```

```
124 definition System  where
125 "System t P r = ((Coordinator 0 (t 0) (P ∪ {0}) r ‖ Channel)
    ↪  ‖ Participants P t)"
```

Wenn es einen Teilnehmer im Committed-Zustand gibt, so muss es eine Commit-Nachricht geben

```
8  definition inv1 where
9    ‹inv1 msgs states ⟷
10     ((∃proc a ack'. proc ≠ 0 ∧ states proc = Committed a
       ↪  ack') ⟶
11                  (∃sender rcpt. (sender, Send rcpt Commit) ∈
                    ↪  msgs))›
```

```
6  definition inv1 where
7    ‹inv1 s ⟷
8      ((∃msgs p. (snd s) p = PState PCommitted msgs) ⟶
9                  (∃sender rcpt. (sender, rcpt, Commit) ∈ snd
                   ↪  (fst s)))›
```

Wenn es eine Commit-Nachricht gibt oder der Koordinator im Committed-Zustand ist, so hat jeder Teilnehmer eine Yes-Nachricht gesendet

```
13  definition inv11 where
14    ‹inv11 msgs states procs ⟷
15      ((∃sender rcpt. (sender, Send rcpt Commit) ∈ msgs) ∨
        ↪  (∃all ack. states 0 = Committed all ack) ⟶
16                  (∀p ∈ procs. p ≠ 0 ⟶ (∃rcpt. (p, Send rcpt
                    ↪  Yes) ∈ msgs)))›
```

```
11  definition inv11 where
12    ‹inv11 procs s ⟷
13      ((∃sender rcpt. (sender, rcpt, Commit) ∈ snd (fst s)) ∨
        ↪  (∃all ack msgs. fst (fst s) = CState (CCommitted all
        ↪  ack) msgs) ⟶
14                  (∀p ∈ procs. p ≠ 0 ⟶ (∃rcpt. (p, rcpt,
                    ↪  Yes) ∈ snd (fst s))))›
```

Wenn der Koordinator im Collecting-Zustand ist, so stimmen hat er sich die korrekten Werte abgespeichert

```
22  definition inv13 where
23    ‹inv13 msgs states procs ⟷
24      (∀t all yes r. states 0 = Collecting t all yes r ⟶
25                  (procs = all ∧ (∀p ∈ yes. p ≠ 0 ⟶ (∃rcpt.
                    ↪  (p, Send rcpt Yes) ∈ msgs))))›
```

```
16  definition inv13 where
17    ‹inv13 procs s ⟷
18      (∀t all yes r msgs. fst (fst s) = CState (Collecting t
        ↪  all yes r) msgs ⟶
19                  (procs = all ∧ (∀p ∈ yes. p ≠ 0 ⟶ (∃rcpt.
                    ↪  (p, rcpt, Yes) ∈ snd (fst s)))))›
```

Wenn der Koordinator im Initial-Zustand ist, so stimmen hat er sich die korrekte Menge an Prozessen abgespeichert

```
27  definition inv14 where
28    ‹inv14 msgs states procs ⟷
29      (∀t all r. states 0 = Initial t all r ⟶ procs = all)›
```

```
21  definition inv14 where
22    ‹inv14 procs s ⟷
23      (∀t all r msgs. fst (fst s) = CState (CInitial t all r)
        ↪  msgs ⟶ procs = all)›
```

Wenn sich der Koordinator im Initial-, Collecting- oder Committed- Zustand befindet, so wurde keine Abort-Nachricht gesendet

```
31  definition inv15 where
32    ‹inv15 msgs states ⟷
33      ((∃t all yes r. states 0 = Initial t all r ∨ states 0 =
        ↪  Collecting t all yes r ∨ states 0 = Committed all
        ↪  yes) ⟶
34              (∀sender rcpt. (sender, Send rcpt Abort) ∉
                ↪  msgs))›
```

```
25  definition inv15 where
26    ‹inv15 s ⟷
27      ((∃t all yes r msgs. fst (fst s) = CState (CInitial t all
        ↪  r) msgs ∨ fst (fst s) = CState (Collecting t all yes
        ↪  r) msgs ∨ fst (fst s) = CState (CCommitted all yes)
        ↪  msgs) ⟶
28              (∀sender rcpt. (sender, rcpt, Abort) ∉ snd (fst
                ↪  s)))›
```

Wenn sich ein Teilnehmer im Aborted-Zustand befindet und keine Abort-Nachricht gesendet wurde, so hat der Teilnehmer auch keine Yes-Nachricht gesendet

```
40  definition inv17 where
41    ‹inv17 msgs states ⟷
42      (∀p. p ≠ 0 ∧ (∃all ack. states p = Aborted all ack) ∧
        ↪  (∀sender recpt. (sender, Send recpt Abort) ∉ msgs)
        ↪  ⟶ (∀rcpt. (p, Send rcpt Yes) ∉ msgs))›
```

```
30  definition inv17 where
31    ‹inv17 s ⟷
32      (∀p. p ≠ 0 ∧ (∃msgs. (snd s) p = PState PAborted msgs) ∧
        ↪  (∀sender recpt. (sender, recpt, Abort) ∉ snd (fst
        ↪  s)) ⟶ (∀rcpt. (p, rcpt, Yes) ∉ snd (fst s)))›
```

Wenn sich der Koordinator im Initial-, Collecting- oder Aborted- Zustand befindet, so wurde keine Commit-Nachricht gesendet

```
52  definition inv110 where
53    ‹inv110 msgs states ⟷
54      ((∃t all yes r. states 0 = Initial t all r ∨ states 0 =
        ↪  Collecting t all yes r ∨ states 0 = Aborted all yes)
        ↪  ⟶
55                (∀sender rcpt. (sender, Send rcpt Commit) ∉
                  ↪  msgs))›
```

```
34  definition inv110 where
35    ‹inv110 s ⟷
36      ((∃t all yes r msgs. fst (fst s) = CState (CInitial t all
        ↪  r) msgs ∨ fst (fst s) = CState (Collecting t all yes
        ↪  r) msgs ∨ fst (fst s) = CState (CAborted all yes)
        ↪  msgs) ⟶
37                (∀sender rcpt. (sender, rcpt, Commit) ∉ snd
                  ↪  (fst s)))›
```

Wenn eine Commit-Nachricht gesendet wurde, so befindet sich der Koordinator entweder im Committed- oder Forgotten-Zustand

```
57  definition inv111 where
58    ‹inv111 msgs states ⟷
59      ((∃sender rcpt. (sender, Send rcpt Commit) ∈ msgs) ⟶
        ↪  (∃all ack. states 0 = Committed all ack ∨ states 0 =
        ↪  Forgotten))›
```

```
39  definition inv111 where
40    ‹inv111 s ⟷
41      ((∃sender rcpt. (sender, rcpt, Commit) ∈ snd (fst s)) ⟶
42       (∃all ack cmsgs. fst (fst s) = CState (CCommitted all
         ↪  ack) cmsgs ∨ fst (fst s) = CState Forgotten cmsgs))›
```

Wenn eine Commit-Nachricht gesendet wurde, so wurde keine Abort-Nachricht gesendet

```
67  definition inv3 where
68    ‹inv3 msgs states ⟷
69      (∃sender recpt. (sender, Send recpt Commit) ∈ msgs) ⟶
        ↪  (∀sender recpt. (sender, Send recpt Abort) ∉ msgs)›
```

```
44  definition inv3 where
45    ‹inv3 s ⟷
46      (∃sender recpt. (sender, recpt, Commit) ∈ snd (fst s))
        ↪  ⟶ (∀sender recpt. (sender, recpt, Abort) ∉ snd (fst
        ↪  s))›
```

Vergleich

"By itself, the I/O automaton model has very little structure, which allows it to be used for modelling many different types of distributed systems. Additional structure must be added to the basic model to enable it to describe particular types of asynchronous systems." Lynch buch s.199

blblblb